Step 2: Constructors in Java ⭐

A **constructor** is a special block that is automatically called when an object is created.

Basic example

```
class Student {

    String name;
    int age;

    Student() {
        System.out.println("Constructor called");
    }

    public static void main(String[] args) {
        Student s1 = new Student();
    }
}
```

Output:

```
Constructor called
```

When you write:

```
Student s1 = new Student();
```

Java calls:

```
Student()
```

automatically.

Types of Constructors

1. Default Constructor

```
class Student {

    Student() {
        System.out.println("Hello");
    }
}
```

2. Parameterized Constructor

```
class Student {

    String name;
```

```
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Create object:

```
Student s1 = new Student("Akshat", 22);
```

Now:

```
name = Akshat
age  = 22
```

⭐ Important Difference

Constructor	Method
Same name as class	Can have any valid name
No return type	Can have return type
Called automatically when object is created	Called explicitly
Initializes objects	Performs operations

Very important MCQ

```
class Test {

    Test() {
        System.out.println("A");
    }

    void Test() {
        System.out.println("B");
    }
}
```

Here:

```
Test()       → Constructor
void Test()  → Method
```

Because **a constructor cannot have a return type**.

Step 3: `this` Keyword in Java ⭐⭐⭐

The `this` keyword refers to the **current object**.

1. Solving variable-name confusion

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Here there are two `name`s:

```
this.name → object's instance variable
name      → constructor parameter
```

So:

```
this.name = name;
```

means:

Put the parameter `name` into the current object's `name`.

Example

```
Student s1 = new Student("Akshat", 22);
Student s2 = new Student("Rahul", 25);
```

Objects:

```
s1 → name = Akshat, age = 22
s2 → name = Rahul, age = 25
```

---

2. `this` can call another method

```
class Student {

    void display() {
        System.out.println("Hello");
    }

    void show() {
        this.display();
    }
}
```

`this.display()` means:

Call `display()` of the current object.

---

3. `this()` calls another constructor

```
class Student {

    Student() {
        this(20);
        System.out.println("Default");
    }

    Student(int age) {
        System.out.println(age);
    }
}
Student s = new Student();
```

Output:

```
20
Default
```

⭐ DSSSB MCQ Trap

`this()` **must be the first statement** inside a constructor.

```
Student() {
    this(20);       // ✅
    System.out.println("Hello");
}
```

Not:

```
Student() {
    System.out.println("Hello");
    this(20);       // ❌ Error
}
```

🧠 Remember

```
this.variable → current object's variable
this.method() → current object's method
this()        → another constructor of same class
```

Next step → `static` keyword in Java.

Step 4: `static` Keyword in Java ⭐⭐⭐

`static` means the member **belongs to the class, not to individual objects**.

1. Static Variable

Without `static`:

```
class Student {
    String college = "ABC";
}
```

Every object gets its **own copy**.

With `static`:

```
class Student {
    static String college = "ABC";
}
```

There is **one shared copy** for the entire class.

Example

```
class Student {

    String name;
    static String college = "ABC";

    Student(String name) {
        this.name = name;
    }

    void display() {
        System.out.println(name + " " + college);
    }

    public static void main(String[] args) {

        Student s1 = new Student("Akshat");
        Student s2 = new Student("Rahul");

        s1.display();
        s2.display();
    }
}
```

Output:

```
Akshat ABC
Rahul ABC
```

Both objects share:

```
static String college;
```

---

2. Static Method

A static method belongs to the **class**.

```
class Test {

    static void show() {
        System.out.println("Hello");
    }

    public static void main(String[] args) {
        Test.show();
    }
}
```

You don't need to create an object.

```
ClassName.method()
```

⭐ Important Rule

A static method **cannot directly access a non-static variable**.

```
class Test {

    int x = 10;

    static void show() {
        System.out.println(x); // ❌ Error
    }
}
```

Why?

Because `x` belongs to an **object**, while `show()` belongs to the **class**.

But this works:

```
class Test {

    static int x = 10;

    static void show() {
        System.out.println(x); // ✅
    }
}
```

---

3. Static Block

A static block executes **when the class is loaded**, before `main()`.

```
class Test {

    static {
        System.out.println("Static block");
```

```
    }

    public static void main(String[] args) {
        System.out.println("Main");
    }
}
```

Output:

```
Static block
Main
```

Multiple static blocks

```
class Test {

    static {
        System.out.println("1");
    }

    static {
        System.out.println("2");
    }

    public static void main(String[] args) {
        System.out.println("3");
    }
}
```

Output:

```
1
2
3
```

They execute **top to bottom**.

🧠 DSSSB Exam Points

Remember these:

```
static variable → one shared copy
static method   → belongs to class
static block    → executes when class loads
```

Very important:

```
static method
     ↓
Can directly access
     ↓
static members ✅
```

```
Cannot directly access
     ↓
non-static members ❌
```

And:

`main()` is static because JVM needs to call it **without creating an object**.

Quick MCQ

What will happen?

```
class Test {

    static int x = 10;

    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();

        t1.x = 20;

        System.out.println(t2.x);
    }
}
```

Answer: `20` ✅

Because `x` is static, both `t1` and `t2` refer to the **same variable**.

Step 5: `final` Keyword in Java ⭐⭐⭐

`final` means **cannot be changed further**.

It can be used with:

1. **Variable**
2. **Method**
3. **Class**

---

1. `final` Variable

Once a value is assigned, it **cannot be changed**.

```
final int x = 10;
```

```
x = 20;   // ❌ Error
```

Think:

```
final variable = constant
```

Example:

```
class Test {
    public static void main(String[] args) {

        final double PI = 3.14159;

        System.out.println(PI);
    }
}
```

---

2. `final` Method

A final method **cannot be overridden** by a child class.

```
class Parent {

    final void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {

    void show() {       // ❌ Error
        System.out.println("Child");
    }
}
```

Why?

Because:

```
final method
     ↓
Cannot be overridden
```

---

3. `final` Class

A final class **cannot be inherited**.

```
final class Parent {

}
```

Now:

```
class Child extends Parent {   // ❌ Error
}
```

Example

```
final class Test {
}
```

You cannot:

```
class Demo extends Test { }   // ❌
```

⭐ Very Important: `final` vs `finally` vs `finalize`

These are commonly asked in exams.

Keyword	Meaning
`final`	Prevents modification/inheritance/overriding
`finally`	Block used with exception handling
`finalize()`	Old GC-related method; deprecated/obsolete in modern Java

Example:

```
final int x = 10;
```

Here `final` prevents reassignment.

🔥 Important MCQ

What happens?

```
final int x;
x = 10;
x = 20;
```

Answer:

```
x = 10;  ✅
x = 20;  ❌
```

A blank `final` variable can generally be assigned **once** before it is used.

🧠 Remember This

```
final variable → Cannot reassign
final method   → Cannot override
final class    → Cannot inherit
```

Easy trick:

Variable → Value can't change
Method → Implementation can't be overridden
Class → Can't be extended